SOC100 — VIDEO 2

Windows OS: Permissions, Processes, Services, Scheduled Tasks, Registry & CMD

Interview-Focused Study Notes — Level Effect SOC100-1

Personal Study Notes

Compiled from course transcript for interview revision

About This Document

These notes are distilled from the full transcript of SOC100-1 Class 2 (“Cybersecurity Analyst Pt 2”), covering Windows user permissions, processes and Task Manager, Windows Services, Scheduled Tasks, installing/removing software, the Windows Registry, and an introduction to the Command Prompt.

Same format as Video 1: stream banter and logistics are stripped out, only technical content and lab walkthroughs remain, each with an interview Q&A section.

New in this version: Screenshot Placeholders

Every practical/hands-on section now includes a green-bordered placeholder marking exactly where a screenshot belongs and what it should show. Rewatch the video, capture the screenshot described, and drop it in at that spot. Theory sections (no hands-on demo) don't get placeholders — only actual lab/demo steps do.

> 📷 **Screenshot Placeholder:** *Example placeholder — this is what they look like. Replace this box's image reference with your own screenshot once captured.*

Topics Covered In This Video

- User permissions: the Security tab, allow/deny, inheritance, and icacls on the command line
- Processes vs. applications vs. services — and Task Manager
- Process IDs (PIDs) — what they are and why you shouldn't over-trust them
- Early indicators of compromise and performance bottlenecks
- Windows Services — the Services console (services.msc)
- Scheduled Tasks — viewing, creating, exporting/importing
- Installing and removing software safely
- The Windows Registry — hives, keys, values
- Registry persistence — the Run key (how malware auto-starts)
- Command Prompt (CMD) fundamentals and file operations
- Automating registry tasks from the command line

1. User Permissions Fundamentals

Concept

Permissions are framed as the bedrock of security — this is where confidentiality begins. A large part of cybersecurity work is preventing unauthorized access while allowing authorized access; permissions, passwords, and encryption all exist to enforce that boundary.

Where permissions live in the GUI

Right-click any file or folder → Properties → Security tab. This shows which users/groups have access and what level of access (checkboxes for each permission type).

Permission levels (Windows-specific, more granular than Linux's read/write/execute)

- Full Control — every other permission combined
- Modify — read, write, and delete, but not full administrative control
- Read & Execute — view and run, no changes
- List Folder Contents — see what's inside a folder
- Read — view only
- Write — add/change content
- Special Permissions — granular permissions not covered by the standard checkboxes; often a sign something was configured via script or automation rather than a normal user/group policy action

A common mistake: clicking Modify or Read & Execute accidentally grants far more than intended, because Windows permissions are grouped (granting Modify silently includes Read & Execute, Read, and Write underneath it).

> 📷 **Screenshot Placeholder:** *Security tab of a folder's Properties window, showing the list of users/groups and their permission checkboxes*

Adding/removing a user's access

Security tab → Edit → Add (type the username, e.g. Guest) → set Allow/Deny per permission → OK. Removing follows the same path: Edit → select the user → Remove.

> 📷 **Screenshot Placeholder:** *Adding the Guest account to a folder's permissions list with Allow checked for Read & Execute*

Inheritance

Inheritance automatically applies a parent folder's permission set down to all subfolders. It simplifies administration (set permissions once at a shared drive's root, everything under it inherits) but is a common real-world source of accidental over-exposure — someone creates a folder under an inherited path and unintentionally exposes it to everyone who had access above it.

Disabling inheritance (Security → Advanced → Disable Inheritance → Remove all inherited permissions) strips everything back to only the permissions explicitly, manually set on that specific object — useful when you want a private folder inside an otherwise shared/inherited structure.

> 📷 **Screenshot Placeholder:** *Advanced Security Settings dialog showing inheritance enabled/disabled state for a folder*

Command-line permissions: icacls

The lecture demonstrates granting/denying access from CMD using icacls, e.g. icacls "Windows PowerShell" /grant Guest:R to grant Guest read access, or /deny to remove it. A permission granted this way shows up as Special Permissions in the GUI — a useful investigative flag that something was set programmatically rather than through normal admin clicking.

> 📷 **Screenshot Placeholder:** *CMD window showing an icacls command being run and its success/failure output message*

Important observed behavior: GUI permission windows are a snapshot, not a live view — changes made via CMD don't appear in an already-open Properties window until it's reopened/refreshed. This is a deliberate design choice (constantly live-updating every open dialog would waste resources) but it's a common point of confusion when troubleshooting.

Interview Q&A

Q: Where do you go to view or change a file/folder's permissions in the Windows GUI?

A: Right-click the file or folder, choose Properties, and go to the Security tab. That's where the list of users/groups and their allowed permissions lives.

Q: What's the difference between Modify and Read & Execute permission levels?

A: Modify is a broader grouped permission that includes the ability to read, write, and delete — it silently includes Read & Execute and Read underneath it. Read & Execute only allows viewing and running, no changes. Because Windows groups permissions this way, checking Modify grants more access than someone might expect if they only meant to allow execution.

Q: What is permission inheritance, and why is it both useful and risky?

A: Inheritance automatically applies a parent folder's permission set to everything underneath it, which makes administering a large folder structure (like a company share drive) much easier — set it once at the top. The risk is that anyone creating a new folder inside that structure can unintentionally expose it to everyone who has access above it, since it inherits those permissions by default unless explicitly changed.

Q: How would you remove inherited permissions from a specific folder so it stops following its parent's permission set?

A: Go to Security → Advanced → Disable Inheritance, then choose to remove all inherited permissions from that object. After that, only permissions explicitly set on that specific folder apply — nothing flows down from the parent anymore.

Q: What does it mean if you see 'Special Permissions' listed on a file instead of a standard permission level?

A: It usually indicates the permission was set through a script, command-line tool (like icacls), or some automated/non-standard process rather than a normal user manually checking boxes in the GUI. It's a useful flag during an investigation — permissions that don't map cleanly to the standard GUI options are worth a closer look.

Q: How would you grant a user read access to a folder from the command line instead of the GUI?

A: Using icacls — for example, icacls "C:\path\to\folder" /grant username:R grants read access. It's the CLI equivalent of checking the Read box on the Security tab, and it's scriptable/repeatable across many objects at once, which the GUI isn't.

Q: Why doesn't a permissions window update automatically when you change something from the command line while it's open?

A: Because GUI dialogs in Windows are largely snapshots in time rather than continuously live-updating views — refreshing every open window in real time would be an unnecessary drain on system resources for information that's rarely needed live. You have to close and reopen (or refresh) the window to see the current state.

2. Processes, Applications, and Task Manager

Definitions

- Application — a compiled, executable bundle of code (.exe) that can be double-clicked/run, like Chrome, Word, or File Explorer.
- Process — an instance of an application actually running. One application can spawn one or many processes depending on how the developer built it (e.g., some apps bundle multiple open windows into a single process; others spawn a new process per window).
- Service — a background process not meant to be interacted with directly by the user (e.g., Print Spooler, Bluetooth support). Services support the OS and other applications rather than being something you click into.

Opening Task Manager

Right-click the taskbar/Start button and select Task Manager, or Ctrl+Alt+Delete on a physical machine (may not work inside a virtual lab session).

> 📷 **Screenshot Placeholder:** *Task Manager open on the Processes tab, showing running applications and their expandable child processes*

Processes tab vs. Details tab

The Processes tab gives a high-level, grouped view (e.g., all of Microsoft Edge's windows/tabs rolled up under one entry with total memory usage). The Details tab gives a much more granular view — every individual process with its own PID, which is essential when you need to identify and act on one specific instance rather than an entire application group.

> 📷 **Screenshot Placeholder:** *Task Manager Details tab showing individual processes with their PID column visible*

Ending a process

Right-click a process → End Task. Demonstrated on Microsoft Edge and Notepad. This is one of the most common first troubleshooting moves for something that's hung or unresponsive.

Performance tab and Resource Monitor

The Performance tab shows CPU, memory, disk, and network/ethernet consumption at a glance. For a more granular view, Resource Monitor (launchable from the bottom of the Performance tab) breaks activity down further, including a live histogram of resource usage over time.

> 📷 **Screenshot Placeholder:** *Task Manager Performance tab showing CPU/memory/network graphs, and Resource Monitor open alongside it*

Connecting processes back to services

Many svchost.exe entries in Task Manager are Service Host processes — a generic host executable that runs one or more underlying Windows services (Local Service or Network Service accounts are a telltale sign). Right-click an svchost process → Go to Services to see exactly which service it corresponds to, which is far easier than trying to match PIDs manually.

> 📷 **Screenshot Placeholder:** *Right-click context menu on an svchost.exe process showing the 'Go to services' option, and the resulting highlighted service*

Interview Q&A

Q: What's the difference between an application, a process, and a service?

A: An application is a compiled executable file you can run, like Chrome or Word. A process is an actual running instance of that application — one app can have one or many processes depending on how it's built. A service is a background process the user isn't meant to interact with directly, supporting the OS or other software rather than being something you click into, like a Print Spooler service.

Q: Why would you use the Details tab instead of the Processes tab in Task Manager?

A: The Processes tab groups everything at a high level — for example, rolling up all of a browser's tabs and windows into one entry. The Details tab breaks things down to individual processes with their own PID, which you need when you have to identify or terminate one specific process rather than an entire application group.

Q: What is svchost.exe, and why are there so many instances of it running?

A: It's the Service Host process — a generic executable Windows uses to run background services rather than giving every single service its own standalone .exe. Many different services can be hosted under different svchost.exe instances, which is why you see so many of them; right-clicking one and choosing 'Go to services' shows exactly which service that particular instance corresponds to.

Q: If a user reports their computer is running slowly, what's your first move in Task Manager?

A: Check the Performance tab for CPU, memory, disk, and network consumption to spot an obvious bottleneck — for example CPU pegged at 99% leaving almost nothing for anything else to run smoothly. From there, sort the Processes/Details view by the resource that's maxed out to identify what's actually consuming it.

3. Process IDs (PIDs)

What a PID is

A Process ID is a numerical value assigned to a running process so it can be referenced precisely — useful when multiple instances of the same executable are running and you need to specify exactly which one to act on (e.g., which svchost.exe, which Notepad window).

Key caution: don't over-trust PIDs

- PIDs are reused — once a process ends, its PID becomes available and Windows can assign it to a different, unrelated process later.
- PIDs are not guaranteed to be consistent across reboots or sessions.
- Malicious activity can involve process injection, where malicious code takes over a normally-trusted process — meaning a PID that looks normal and expected may no longer actually be running trustworthy code.

Practical guidance from the lecture: treat a PID purely as a referenceable value to track down and act on a specific process — not as a trust indicator on its own.

Interview Q&A

Q: What is a PID and why is it useful?

A: A Process ID is a unique numerical identifier assigned to a running process, letting you reference one specific instance precisely — critical when several instances of the same program are running and you need to know exactly which one to inspect or terminate.

Q: Should you trust a PID as proof that a process is legitimate?

A: No. PIDs are reused once a process ends, and they aren't guaranteed to stay consistent across reboots. More importantly, process injection techniques can let malicious code run inside what looks like a normal, trusted process — so a familiar-looking PID doesn't guarantee the process is actually trustworthy. A PID should be treated as a reference value, not a trust signal.

4. Early Indicators of Compromise & Performance Bottlenecks

What to look for as a beginner

Without deep triage tooling yet, the lecture's practical advice is to first learn what normal looks like — what processes should typically be running for a given user context — so that something abnormal (an unexpected CMD, PowerShell, or unfamiliar background process appearing) stands out as a visual cue.

- Strange performance bottlenecks — CPU, memory, or network spiking unexpectedly relative to what the user is actually doing (e.g., opening a plain Excel file shouldn't cause a network spike).
- “Beaconing” style network activity — traffic going up and down in a repeated, regular pattern rather than normal organic usage.
- Processes running outside the context of the current user.

At this beginner stage, relying on Windows Defender/Firewall to flag known-bad activity is reasonable — deeper manual triage (e.g., using the Sysinternals suite and Process Monitor to inspect individual system API calls between user space and kernel space) comes later in the course.

Bottleneck concept

Analogy used: a highway with too many cars — once a resource (CPU, memory, network) is maxed out, everything else backs up behind it. If CPU is at 99%, only 1% remains to share across everything else the system needs to do, and the system grinds to a halt.

Interview Q&A

Q: As a beginner without advanced tooling, how would you first notice a potential compromise on a Windows machine?

A: By first understanding what's normal for that user/context, then watching for what doesn't fit — an unexpected CMD or PowerShell window running in the background, unusual processes outside the current user's normal activity, or performance spikes that don't match what the user is actually doing, like a plain Excel file causing a network spike.

Q: What is a performance bottleneck, in plain terms?

A: It's when one resource — CPU, memory, or network — gets maxed out and becomes the limiting factor for everything else on the system, similar to too many cars merging onto a highway and backing up all following traffic. If CPU sits at 99%, there's only 1% left for every other process competing for it, so the whole system slows to a crawl.

Q: What does 'beaconing' network activity mean, and why is it suspicious?

A: It refers to network traffic that goes up and down in a regular, repeated pattern rather than organic, human-driven usage — often a sign of malware checking in with a command-and-control server at set intervals rather than normal application behavior.

5. Technical Note-Taking

Practical advice given rather than a hard rule: keep running notes as you go through technical material, since VM labs can be reset and progress lost. Suggested tools include Notion, Obsidian, or even pen and paper — the specific tool matters less than the habit. One security-flavored consideration raised: Obsidian stores notes locally with no vendor access, which some security-minded users prefer over cloud tools whose terms of service may permit vendor access for support purposes. Screenshots plus written notes together were recommended as a stronger combination than either alone.

6. Windows Services

Concept

A service is a background process, generally not something a regular user starts or interacts with directly — it should be scheduled or triggered by something else in the system rather than manually launched.

The Services console (services.msc)

Opened via Start menu search, or Win+R → services.msc. Lists every service Windows is configured with, whether currently running or not, along with its logon account and Startup Type.

> 📷 **Screenshot Placeholder:** *Services console (services.msc) with the Status column sorted, showing a mix of running and stopped services*

Startup types

- Automatic — starts on boot, Windows manages priority/queue order
- Automatic (Delayed Start) — starts a short time after boot, once the system is less busy
- Manual (Trigger Start) — only starts when triggered by a specific event or action
- Disabled — will not start under any circumstance until re-enabled

Managing a service

Right-click a service (e.g., Bluetooth Support Service) → Start / Stop / Restart / Pause / Resume. Double-click (or Properties) shows the full description, the path to the executable it calls, logon account, recovery options (what to do on 1st/2nd/subsequent failure), and any dependencies on other services or system components.

> 📷 **Screenshot Placeholder:** *Properties dialog of a service showing its executable path (e.g., pointing to svchost.exe) and Startup Type dropdown*

Why this matters for security work

Custom/third-party software (Oracle, Java, company-internal apps) frequently installs its own service — and this is also a known place malware can hide, configuring itself to run as a service that starts automatically and blends in among dozens of legitimate entries.

Interview Q&A

Q: What is a Windows service, and how is it different from a regular application process?

A: A service is a background process that isn't meant to be launched or interacted with directly by the user — it runs to support the OS or other applications, like Bluetooth support or a Print Spooler. A regular application process is something the user directly opens and interacts with, like a browser window.

Q: What's the difference between Automatic, Automatic (Delayed Start), and Manual startup types?

A: Automatic starts the service at boot, prioritized by Windows in its own startup queue. Automatic (Delayed Start) intentionally waits until shortly after boot once the system is less busy, to avoid

overloading startup. Manual (Trigger Start) means the service only starts when something specific triggers it — it won't run just because the system booted.

Q: Where would you look to find the executable path and dependencies of a specific Windows service?

A: Open services.msc, right-click (or double-click) the service, and go to Properties — the General tab shows the path to the executable it calls, and the Dependencies tab shows what other services or components it relies on.

Q: Why are Windows services relevant to malware investigation?

A: Because malware can configure itself to run as a service that starts automatically at boot, which lets it blend in among the dozens of legitimate services already present and gives it persistence without the user manually launching anything. Reviewing services for anything unfamiliar or unexpectedly added is part of basic endpoint triage.

7. Scheduled Tasks

Concept

Scheduled Tasks let Windows automate running something at a specific time or in response to a trigger — a backup job at 11pm, a patch/update at 2-3am, or a program that starts every time a specific user logs in.

Opening Task Scheduler

Start menu search → Task Scheduler, or Win+R → taskschd.msc.

> 📷 **Screenshot Placeholder:** *Task Scheduler console showing the Task Scheduler Library with existing default tasks listed*

Reviewing existing tasks

Even a fresh workstation has some default scheduled tasks — the lecture's example was a Microsoft Edge update task (fires nightly) and an npcap Watchdog task, which turned out to belong to Wireshark's installer (npcap is Wireshark's packet-capture driver component; its installer schedules a watchdog task to make sure the loopback adapter stays enabled). This models a real workflow: when you don't recognize a scheduled task, look it up by name rather than guessing.

> 📷 **Screenshot Placeholder:** *Properties of an existing scheduled task (e.g., the npcap Watchdog task) showing its Triggers and Actions tabs*

Creating a basic task

Action pane → Create Basic Task. Walkthrough: name it, choose a trigger (e.g., “When I log on”), choose the action (“Start a program”), and browse to or type the path to the target executable (the lecture located PowerShell's actual path by right-clicking its taskbar shortcut → Properties, then confirmed it via Task Manager → right-click process → Open File Location).

> 📷 **Screenshot Placeholder:** *Create Basic Task wizard showing the trigger selection screen (e.g., 'When I log on')*
>
> 📷 **Screenshot Placeholder:** *Create Basic Task wizard showing the action/program path screen with the target executable selected*

Testing, exporting, and importing tasks

Right-click a created task → Run to test it immediately. Tasks can be exported to an XML file (Export) for backup, sharing, or offline inspection, and re-imported later (Import). The exported XML contains registration info, author, user ID, and configuration — useful during forensic investigation of a scheduled task suspected of being malicious, since it can be examined before deciding whether to delete it.

> 📷 **Screenshot Placeholder:** *Exported scheduled task .xml file opened in a text editor, showing its structure (registration info, triggers, actions)*
>
> 📷 **Screenshot Placeholder:** *Import Task dialog showing the option to change user/group, run whether logged in or not, and run with highest privileges*

Interview Q&A

Q: What is a scheduled task used for, and how would you create one to run a program at logon?

A: Scheduled Tasks automate running a program or script at a specific time or in response to a trigger, like a login event, a specific time of day, or system startup. To create one: open Task Scheduler, Create Basic Task, choose a trigger such as 'When I log on', choose 'Start a program' as the action, and provide the path to the target executable.

Q: You find an unfamiliar scheduled task on a workstation. What's your process for figuring out if it's legitimate?

A: Search for the task's exact name to see if it belongs to known, legitimate software — the lecture's example was an 'npcap Watchdog' task that turned out to belong to Wireshark's packet-capture driver rather than being suspicious. If it doesn't map to anything recognizable, exporting it to XML lets you inspect its registration info, author, and configured action in more detail before deciding whether to investigate further or remove it.

Q: Why would you export a scheduled task to XML during an investigation instead of just deleting it?

A: Because the exported XML preserves the task's full configuration — registration info, author, triggers, and the exact action it was set to perform — which can be analyzed for indicators of where it came from and what it was doing, before you destroy that evidence by deleting the task outright.

Q: What options are presented when importing a scheduled task, and why do they matter?

A: You can choose which user/group the task runs as, whether it runs whether the user is logged in or not, and whether it runs with highest privileges. These matter because a task configured to run with elevated privileges regardless of login state is far more capable (and, if malicious, more dangerous) than one limited to a specific logged-in user's context.

8. Installing and Removing Software

Installing

As an administrator, installing software is generally just downloading and double-clicking the installer. Windows shows more configuration prompts during install than macOS typically does (which is often just drag-and-drop a package).

> 📷 **Screenshot Placeholder:** *Notepad++ installer welcome screen and license agreement (EULA) step*

Things to watch for during install

- License Agreement (EULA) — practically speaking, almost no one reads it in full; it's a liability waiver and usage terms, and by agreeing you accept responsibility for how you use the software.
- Bundled software / pre-checked extra installs — historically a common vector for adware/spyware (the lecture references older bundled antivirus trialware like Avast/AVG/McAfee/Norton as an example of software that used to ride along with installers). Read the checkboxes on custom/component-selection screens rather than blindly clicking Next.
- File association overrides — some installers (Adobe products are called out specifically) try to set themselves as the default handler for file types like PDF or DOCX; watch for an unchecked/checked box controlling this.
- Post-install update prompts — generally safe to accept unless you're a sysadmin/developer who deliberately needs to stay on a specific version.

> 📷 **Screenshot Placeholder:** *Custom/component selection screen during install showing checkboxes for optional components or bundled offers*

Recommended endpoint protection (as discussed)

- Windows Defender (built-in antivirus/firewall) — described as more than sufficient for most home users, consistently rated well in independent testing, and free.
- A browser ad/script blocker (uBlock Origin was named specifically) as arguably the single highest-value protective layer, since most initial compromise happens through email links and malicious web content rather than a user deliberately downloading and running malware.
- Malwarebytes mentioned as a reasonable additional scanning layer if wanted (cross-platform, including Mac).

Removing software

Two paths: right-click the app in the Start menu → Uninstall, or the traditional route through Add/Remove Programs (Settings → Apps, in modern Windows) → locate the app → Uninstall. Both trigger the same underlying uninstall routine; a reboot is sometimes required afterward.

> 📷 **Screenshot Placeholder:** *Add/Remove Programs (Settings > Apps) list with an installed application selected and the Uninstall button visible*

Interview Q&A

Q: What should you watch for during a typical Windows software installation to avoid installing unwanted extras?

A: Read the component/custom-install screen carefully rather than clicking Next by default — this is historically where bundled adware, trial antivirus software, or other unwanted add-ons get pre-checked for installation. Also watch for file-association override checkboxes, where an installer tries to make itself the default handler for file types like PDF.

Q: What's the built-in Windows Defender's reputation for home/basic protection, based on this lecture?

A: It's considered more than sufficient for most home users and consistently scores well in independent annual testing against various malware infection vectors — it's free, built in, and doesn't require a third-party purchase for baseline protection.

Q: According to the lecture, what's actually the highest-value protective layer for an average user, even above antivirus?

A: A browser-level ad/script blocker like uBlock Origin — because the most common initial access vector is email links and malicious web content a user clicks on, not a user knowingly downloading and running an .exe. Blocking malicious redirects and scripts at the browser level addresses the attack path before it ever reaches the point where antivirus would need to catch it.

Q: What are the two ways to uninstall a program in Windows?

A: Right-click the application directly in the Start menu and choose Uninstall, or go through Settings → Apps (the modern equivalent of the old Add/Remove Programs control panel), locate the application in the list, and choose Uninstall. Both trigger the same uninstall routine.

9. Windows Registry Fundamentals

Concept

Simplest mental model given: imagine a giant Excel spreadsheet/database with many tables, rows, and columns linking to one another. The registry is a giant key-value store — for any given setting, configuration, path, or preference the OS or an application needs to remember, there's very likely a corresponding entry somewhere in the registry.

Danger framing: connecting back to the kernel/engine analogy from Video 1 — editing the registry is like reaching into the engine itself and changing a setting (e.g., the temperature at which it should trigger cooling). It's powerful and can affect how the kernel directs user-space processes, so it should be approached with caution, not casual experimentation.

What is and isn't in the registry

If a value isn't in the registry, it typically exists either in a configuration file, or as a runtime/ephemeral value that only exists while something is actively running (PIDs are the example given — they're never stored in the registry or a config file; they exist only for the duration a process is active, the same conceptual category as ephemeral network ports covered later in networking).

Opening the Registry Editor

Start menu search → “registry” or “regedit” → Registry Editor.

> 📷 **Screenshot Placeholder:** *Registry Editor open, showing the top-level hive tree in the left-hand pane*

Interview Q&A

Q: In simple terms, what is the Windows Registry?

A: It's a giant key-value database of configuration information — think of it like a massive spreadsheet with rows and columns linking values together. Practically anything Windows or an installed application needs to remember (a setting, a file path, a preference) is very likely stored as an entry somewhere in the registry.

Q: Why is editing the registry considered risky?

A: Because the registry directly informs how the kernel configures and directs user-space processes and hardware behavior — changing a value incorrectly is like reaching into a car's engine and altering a setting it depends on to run correctly. A wrong or careless edit can cause instability or break functionality, so it's approached with caution rather than casual experimentation.

Q: If a piece of information isn't in the registry, where else might it live?

A: Either in a dedicated configuration file, or as a runtime/ephemeral value that only exists while a process is actively running and is never persisted anywhere — Process IDs (PIDs) are the clearest example: they exist only for the life of the running process and are never written to the registry or any config file.

10. Registry Structure: Hives and Root Keys

Terminology

HKEY stands for Handle to a Key (legacy naming convention) — commonly abbreviated as HK (e.g., HKCU, HKLM). Individual groupings of related key-value pairs are called registry hives — the terminology exists because it's a less rigid, more organic structure than a traditional single database, similar to a beehive holding many individual cells of data.

The main root keys

Root Key	What it holds
HKEY_CLASSES_ROOT (HKCR)	File extension associations — which program opens which file type (e.g., .txt → Notepad)
HKEY_CURRENT_USER (HKCU)	Settings/preferences for the currently logged-in user only — desktop layout, personal startup programs, folder preferences
HKEY_LOCAL_MACHINE (HKLM)	Machine-wide settings applying to all users regardless of who's logged in — the most dangerous place for malware to plant persistence, since it isn't limited to one user account
HKEY_USERS (HKU)	A mirror/reference area holding information across all user profiles on the machine
HKEY_CURRENT_CONFIG (HKCC)	Current hardware configuration — printer, Bluetooth, network adapter, memory/CPU-related info

Practical framing repeated twice in the lecture: nobody memorizes the full registry structure. What matters is understanding the general concept (the registry stores config for a user, the machine, or the hardware) well enough to know where to look and how to search documentation when troubleshooting or configuring something specific. HKCU, HKLM, and HKCC are named as the three most frequently referenced in day-to-day sysadmin/analyst work.

Interview Q&A

Q: What does HKEY stand for, and what's a registry hive?

A: HKEY stands for Handle to a Key — it's legacy naming convention. A registry hive is a grouping of related key-value pairs, conceptually similar to a section of the larger registry database; the 'hive' terminology reflects its organic, grouped structure rather than a single rigid table.

Q: What's the practical difference between HKEY_CURRENT_USER and HKEY_LOCAL_MACHINE?

A: HKCU holds settings specific to whichever user is currently logged in — personal preferences, per-user startup items. HKLM holds machine-wide settings that apply regardless of who's logged in. That distinction matters a lot for malware persistence: something planted in HKLM affects every user of the machine, not just one account, making it a more dangerous and higher-value target for an attacker than HKCU.

Q: Do SOC analysts memorize every registry key path?

A: No — the lecture is explicit about this. Nobody memorizes the full registry structure; what matters is understanding the general concept of what's stored where (user-specific vs. machine-wide vs. hardware config) well enough to know where to start looking, and then referencing documentation for the exact path needed in a specific situation.

Q: **Which root key would you check to see what program opens .txt files by default?**

A: HKEY_CLASSES_ROOT (HKCR) — it holds file extension associations, mapping file types to the programs registered to handle them.

11. Registry Persistence — The Run Key (Malware Auto-Start)

Why this matters

This is presented as one of the most practically important things covered in the whole session — a Tier 1/help desk analyst who knows to check this Run key location already knows something many working help desk staff don't.

The path

HKEY_CURRENT_USER \ Software \ Microsoft \ Windows \ CurrentVersion \ Run — this is the per-user startup Run key. Windows checks this location (and its HKLM equivalent, for all-user persistence) on boot/login and launches anything listed there.

> 📷 **Screenshot Placeholder:** *Registry Editor navigated to HKCU\Software\Microsoft\Windows\CurrentVersion\Run*

Live demo walkthrough

- Right-click inside the Run key → New → String Value — named it 'start_notepad' with the value pointing to notepad.exe (labeled 'malicious notepad' for the demo, standing in for how real malware would configure the exact same mechanism).
- Exported the key/value to a .reg file on the desktop for inspection.
- Opened the exported .reg file in a text editor — it's a plain text registry script showing the target hive path (HKCU\Software\Microsoft\Windows\CurrentVersion\Run) and the value. Note: paths in .reg files use double backslashes (\\) as an escape sequence so the backslash character is interpreted literally rather than as a special character.
- Deleted the value from the registry, then re-imported the .reg file to demonstrate that double-clicking a .reg file silently re-adds the entry — this is exactly how old-style drive-by/downloaded malware used to establish persistence.
- Rebooted the VM — Notepad launched automatically on login, proving the persistence mechanism works.
- Removed the entry afterward to clean up.

> 📷 **Screenshot Placeholder:** *New String Value being created inside the Run key via right-click > New > String Value*
>
> 📷 **Screenshot Placeholder:** *Exported .reg file opened in a text/notepad editor, showing the registry path and escaped backslashes*
>
> 📷 **Screenshot Placeholder:** *Desktop/File Explorer showing the exported .reg file with its icon and .reg extension visible (View > File Name Extensions enabled)*
>
> 📷 **Screenshot Placeholder:** *Notepad automatically launching immediately after a VM reboot/login, demonstrating the Run key persistence*

Real-world triage relevance

If a user reports something launching every time they log in that they didn't intentionally set up, or a file that should open one way (e.g., an Excel file) behaves unexpectedly, the Run key is one of the first places a help desk/Tier 1 analyst should check. This applies to legitimate troubleshooting too — e.g., applying a vendor-recommended patch for old Java/Oracle/Adobe software sometimes requires a direct registry edit.

Interview Q&A

Q: Where would you look in the registry to find a program configured to auto-start when a specific user logs in?

A: HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run — Windows checks this Run key on login and launches anything listed there. There's an HKEY_LOCAL_MACHINE equivalent for programs that should start for every user on the machine, regardless of who logs in.

Q: Walk me through how you'd add a program to auto-start via the registry.

A: Navigate to the Run key path in Registry Editor, right-click in the right-hand pane, choose New → String Value, give it a name, and set its value data to the full path of the target executable. On the next login, Windows reads that key and launches the specified program automatically.

Q: Why do exported .reg files often show double backslashes in file paths?

A: Because the backslash is a special/escape character in that file format, so a literal single backslash in a path has to be written as two backslashes (\\) for it to be interpreted correctly as a plain backslash rather than triggering an escape sequence.

Q: How does this Run key relate to real-world malware behavior?

A: It's exactly the mechanism a lot of real persistence malware uses — a downloaded or drive-by payload writes an entry into this same Run key (often in HKLM for machine-wide persistence rather than just one user) so that it automatically relaunches every time the machine boots or the user logs in, without needing the user to manually run it again.

Q: A user says a program starts automatically every time they log in and they don't know why. What's your first troubleshooting step?

A: Check the Run key locations in the registry (HKCU and HKLM under Software\Microsoft\Windows\CurrentVersion\Run) for an entry pointing to that program. This is a fast, well-known place to identify and remove unwanted startup behavior, whether it's legitimate leftover software configuration or something malicious.

12. Command Prompt (CMD) Fundamentals

Background

CMD traces back to the original MS-DOS command-line interface — Windows has kept it around because it's still effective, lightweight, and in some cases faster/more direct than the GUI for the same operation. PowerShell (covered in the next video) is described as a purpose-built scripting language designed specifically to interact with Windows APIs, domain/Enterprise networking, and the .NET Framework — more powerful than CMD but with its own learning curve, covered separately.

Direct kernel interaction example

A CMD command like shutdown sends a direct signal to the kernel to power down the hardware — functionally the same end result as clicking through Start → Power → Shut down in the GUI, but the CLI version can be parameterized (delay timers, restart vs. hibernate vs. shutdown, forced vs. graceful) in ways the GUI's three basic buttons can't match.

> 📷 **Screenshot Placeholder:** *CMD window with the shutdown command and its available parameters/help output*

Interview Q&A

Q: Where does CMD come from, and why does Windows still include it?

A: It traces back to the original MS-DOS command-line interface. Windows has kept it around because it's lightweight, still fully functional, and in many cases faster and more scriptable than achieving the same result through the GUI, even though PowerShell has since become the more powerful, purpose-built option for deeper Windows administration.

Q: Give an example of a CMD command that interacts directly with the kernel, and explain why the CLI version is more flexible than the GUI equivalent.

A: The shutdown command sends a direct instruction to the kernel to power down the hardware. The GUI equivalent (Start → Power → Shut down) only offers a few fixed options, while the CLI version supports parameters for things like delayed shutdown, forced closure of applications, or choosing between restart/hibernate/shutdown — far more control in a single line than clicking through a limited GUI menu.

13. CMD vs. GUI Management

The GUI is good for learning and occasional/one-off tasks — it's discoverable, visual, and doesn't require memorizing syntax. The CLI wins on customization, speed, and repeatability, especially once a task needs to be automated or run more than once. The lecture's stated rule of thumb: if you're doing something once, the GUI is fine; if you're doing it twice in a short window or expect to repeat it, start thinking about scripting it instead.

Interview Q&A

Q: When would you choose the GUI over the command line for a Windows administration task, and vice versa?

A: GUI for a one-off, occasional task where discoverability matters more than speed — you don't need to remember exact syntax. Command line once a task needs to be repeated, automated, or chained together with other actions, since it's faster, scriptable, and far more customizable than clicking through fixed GUI options.

14. File and Directory Operations in CMD

Core commands demonstrated

Command	What it does
dir	Lists contents of the current directory
cd <folder>	Changes into a subfolder (Tab-completes folder names)
cd ..	Moves up one directory level
cd "folder with spaces"	Quotes required around any path containing spaces
mkdir <name>	Creates a new directory (updates the GUI view in real time)
echo <text>	Prints text to the terminal
echo <text> > file.txt	Redirects (writes/overwrites) text into a new or existing file
echo <text> >> file.txt	Redirects and appends text to a file instead of overwriting it
type file.txt	Outputs a file's contents to the terminal (read without opening an editor)
rename old.txt new.txt	Renames a file
del / rm	Deletes a file
cls	Clears the terminal screen
tasklist	Command-line equivalent of Task Manager's process list
taskkill /PID <pid>	Terminates a specific process by its PID
ver	Shows the current Windows version/build number

Notes and habits worth keeping

- Tab-completion: pressing Tab auto-completes folder/file names; Shift+Tab cycles backward through options.
- Ctrl+C cancels/breaks out of a running or malformed command.
- The Up arrow key recalls previous commands from history.
- ss64.com was mentioned as a reference site for looking up CMD command syntax — nobody memorizes every command, the same philosophy given for the registry.

> 📷 **Screenshot Placeholder:** *CMD window demonstrating dir, cd with Tab-completion, and mkdir creating a new folder visible in File Explorer simultaneously*

> 📷 **Screenshot Placeholder:** *CMD window showing echo redirected into a file with >, then type used to read it back*
>
> 📷 **Screenshot Placeholder:** *CMD window showing tasklist output and a successful taskkill /PID command output*

Interview Q&A

Q: How do you list the contents of the current directory and move into a subfolder using CMD?

A: dir lists the contents of the current directory. cd <foldername> moves into a subfolder — Tab-completion can autocomplete the folder name as you type, and cd .. moves back up one level.

Q: How would you create a new text file with specific content directly from CMD?

A: Using the redirect operator with echo — for example, echo Hello World > notes.txt creates (or overwrites) notes.txt with that text. Using >> instead of > appends to the file instead of overwriting its existing contents.

Q: How do you view a file's contents in CMD without opening a text editor?

A: The type command — for example, type notes.txt prints the file's contents directly to the terminal.

Q: How would you terminate a specific hung process from the command line if you already know its PID?

A: taskkill /PID <the process ID> — for example, taskkill /PID 3160. This is the CLI equivalent of right-clicking a process in Task Manager and choosing End Task, but scriptable and precise when you already know exactly which PID you're targeting.

Q: What's the CLI equivalent of Task Manager's process list?

A: tasklist — it prints out the running processes similarly to what you'd see in Task Manager, and pairs naturally with taskkill for identifying and then terminating a specific process by PID from the same terminal session.

15. Automating Registry Tasks via CMD

Core commands demonstrated

Command	What it does
reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"	Reads and displays the current values under a given registry key
reg delete "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v start_notepad /f	Deletes a specific value from a registry key; /f forces it without a confirmation prompt

Demonstrated as the CLI equivalent of everything done manually in Registry Editor earlier — querying the Run key to confirm the malicious notepad entry was present, then deleting it with reg delete instead of navigating and right-clicking in the GUI. Same end result, achieved faster and in a form that can be scripted and repeated across many machines.

> 📷 **Screenshot Placeholder:** *CMD window showing a reg query command returning the Run key's current values, followed by a reg delete command removing the entry*

Interview Q&A

Q: How would you check what's currently configured in a specific registry Run key using CMD instead of Registry Editor?

A: reg query, pointing at the full key path — for example, reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" lists the current values under that key directly in the terminal, without opening Registry Editor.

Q: How would you remove a specific malicious startup entry from the registry using the command line?

A: reg delete, targeting the key path and the specific value name, with /f to force the deletion without a confirmation prompt — for example, reg delete "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v start_notepad /f.

Q: Why would a SOC analyst prefer reg query/reg delete over manually navigating Registry Editor during an incident?

A: Speed and repeatability — a single command can check or remove a specific value instantly and can be scripted to run identically across many endpoints during a wider incident response, whereas manually navigating Registry Editor's tree structure is slower and doesn't scale past one machine at a time.

Quick-Reference Glossary

Term	Definition
icacls	CMD utility for viewing and modifying file/folder permissions
Inheritance	Automatic pass-down of a parent folder's permissions to everything beneath it
PID	Process ID — a numerical, reusable, non-guaranteed-consistent reference to one running process instance
svchost.exe	Generic Service Host executable that runs one or more background Windows services
Service	A background process not meant for direct user interaction; can be Automatic, Delayed, Manual, or Disabled
Scheduled Task	An automation entry that runs a program based on a time or event trigger
Registry	Windows' key-value configuration database for the OS, hardware, and applications
Hive	A grouping of related registry key-value pairs
HKCU / HKLM / HKCC / HKCR / HKU	The five main registry root keys — current user, local machine, current hardware config, file associations, and all-users mirror, respectively
Run key	HKCU or HKLM \Software\Microsoft\Windows\CurrentVersion\Run — where auto-start programs (legitimate or malicious) are configured
.reg file	A plain-text registry script that, when run, writes its contents directly into the registry
reg query / reg add / reg delete	CMD commands for reading, creating, and removing registry values without opening Registry Editor

End of Video 2 notes. Once screenshots are added at each placeholder, this becomes a complete lab record you can walk an interviewer through step by step. Next: Video 3 — PowerShell, Intro to Linux.